

Implementasi Parallel Merge Sort menggunakan OpenMPI

Fairuz Apuilla Rahagi
13222107
Teknik Elektor
Institut Teknologi Bandung
29 April 2026

1 Pendahuluan

Laporan ini menyajikan hasil implementasi dan evaluasi algoritma *sorting Parallel Merge Sort* menggunakan pustaka *Message Passing Interface* (OpenMPI) dan bahasa C++ untuk memperhatikan bagaimana pengaruh komputasi paralel. Evaluasi dilakukan untuk mengukur efisiensi kinerja algoritma komputasi paralel dibandingkan dengan eksekusi sekuensial pada berbagai ukuran set data masukan.

2 Metodologi Eksekusi

Program dirancang dengan arsitektur *Root-Worker* dengan alur komputasi sebagai berikut:

1. **Distribusi (Scatter):** *Root* (Rank 0) membaca set data (berisi rentang 10^4 hingga 10^6 elemen *floating-point* acak) dan menyebarkan potongan *array* secara merata kepada seluruh prosesor yang tersisa menggunakan fungsi `MPI_Scatterv`.
2. **Komputasi Paralel:** Setiap prosesor secara independen mengurutkan datanya masing-masing menggunakan algoritma *Introsort* (`std::sort`), dimana digunakan *library* C++ yang sudah tersedia
3. **Penggabungan (Gather & Merge):** Data yang telah terurut dikumpulkan kembali ke proses *Root* menggunakan `MPI_Gatherv`, disusul dengan fase penggabungan akhir secara sekuensial menggunakan `std::inplace_merge`.

Spesifikasi dari perangkat keras yang digunakan merupakan sebagai berikut:

- **Prosesor:** Intel(R) Core(TM) i9-11900H (Generasi ke-11 Tiger Lake, 8 *Physical Cores*, 16 *Logical Threads*).
- **Compiler:** MSVC (Microsoft Visual Studio), Mode *Release*.

3 Hasil Pengujian

Pengujian kinerja dilakukan menggunakan skenario eksekusi dengan 1, 2, 4, dan 8 prosesor pada tiga variasi ukuran set data. Tabel 1 menunjukkan waktu komputasi murni yang diukur menggunakan fungsi `MPI_Wtime()`.

Table 1: Perbandingan Waktu Eksekusi (dalam detik)

Ukuran Data	1 Prosesor	2 Prosesor	4 Prosesor	8 Prosesor
10.000 elemen	0.004724	0.003561	0.002025	0.003823
100.000 elemen	0.008345	0.005633	0.006830	0.005977
1.000.000 elemen	0.144879	0.058076	0.045853	0.039539

4 Analisis

Berdasarkan waktur yang terukur pada Tabel 1, didapatkan analisis mengenai komputasi paralel yang terjadi:

4.1 *Communication Overhead* pada Beban Rendah

Pada set data berukuran kecil (10.000 dan 100.000 elemen), penambahan jumlah prosesor (misalnya dari 4 ke 8 prosesor) justru menunjukkan penurunan performa. Waktu eksekusi meningkat bahkan melebihi eksekusi dengan menggunakan 8 prosesor. Hal ini disebabkan oleh biaya *communication overhead*, di mana waktu latensi yang dihabiskan untuk mengatur komunikasi antar-proses MPI (seperti *scatter* dan *gather*) mendominasi dan melebihi waktu komputasi pengurutan itu sendiri.

4.2 Efek *Cache* dan Skalabilitas Optimal

Ketika data set ditingkatkan menjadi 1.000.000 elemen, algoritma paralel terbukti efektif. Terjadi percepatan linear yang drastis dari 1 prosesor (0.144 detik) ke 2 prosesor (0.058 detik). Penurunan waktu eksekusi hingga ~ 2.49 kali lipat ini merupakan indikasi kuat dari fenomena efisiensi hierarki memori, di mana pemecahan *array* berskala besar menjadi blok-blok memori yang lebih kecil memungkinkannya untuk ditampung secara optimal di dalam *Cache L2/L3* prosesor.

4.3 *Bottleneck Sorting* Secara Sekuensial

Meskipun terjadi peningkatan performa secara keseluruhan pada set data paling besar, efisiensi cenderung menurun pada saat menggunakan 8 prosesor. Hal ini disebabkan oleh karena proses yang paling sekuensial pada saat melakukan *Introsort* (`std::sort`). Fenomena ini juga terjadi pada tugas sebelumnya pada saat menggunakan openMP dan dapat terlihat pembebanannya di Intel V-tune Profiler

5 Kesimpulan

Implementasi *Parallel Merge Sort* berbasis OpenMPI berhasil mendistribusikan dan menyelesaikan komputasi secara valid dan efisiensi waktunya meningkat. Efisiensi komputasi terdistribusi baru akan optimal jika rasio beban komputasi lebih besar jika dibandingkan dengan biaya /textitoverheadnya. Untuk optimasi tingkat lanjut, implementasi penggabungan berstruktur (*Tree-based Parallel Merge*) dapat direkomendasikan untuk mengurangi *bottleneck* sekuensial di proses `std::sort` pada *Root*.